Notes

Designing Large Scale AI Systems

a.y. 2025/2026

Davide Donà - 264467 Andrea Blushi - 264468

April 2026

Contents

1 Introduction **1**

1.1 Large Language Models (LLMs) 1

1.2 Software 3.0 1

1.3 Roles of AI 1

1.4 Challenges 2

2 Prompting **3**

2.1 Anatomy of a Prompt 3

2.2 API Parameters 4

2.3 Prompting Strategies 4

2.3.1 Prompt Structure 5

2.3.2 Few-shot prompting 5

2.3.3 Chain-of-thought prompting 6

3 Stateful Agents **7**

3.1 Stateless 7

3.2 Default Stateful Agents 8

3.3 Full History 8

3.4 Sliding Window 8

3.5 Summarization 9

3.6 Long-Term Memory 9

3.7 When to use which pattern 10

4 Agent Tools **11**

4.1 Register a Tool 11

4.2 Multi-Tool loop 12

4.3 Problems with tool use 12

5 Evaluation **14**
5.1 Software 1.0 - Evaluation in Traditional Software Development . 14
5.2 Software 2.0 - Evaluation in Machine Learning 14
5.2.1 Comparing Models 15
5.2.2 Data Splitting 15
5.2.3 Experiment Design 17
5.3 Software 2.0 - Statistical Testing 18
5.3.1 True metrics vs Estimated metrics 18
5.3.2 Randomness 20
5.3.3 Evaluation Metrics - Regression 21
5.3.4 Evaluation Metrics - Classification 22
5.4 No Free Lunch Theorem 25

6 Experiment Design **27**
6.1 Introduction 27
6.2 Basic, Probabilities and Random Variables 27
6.2.1 Definition of Probability 27
6.2.2 Axioms of Probability 28
6.2.3 Random Variables 28
6.3 Foundational Random Variables 29
6.3.1 Binomial Random Variable 29
6.3.2 Bernoulli Random Variable 30
6.3.3 Other PMFs Random Variables 30
6.4 Mastering Uncertainty 31
6.4.1 Conditional Probability 31
6.4.2 Expectation 31
6.4.3 Variance 32
6.4.4 Law of Total Probability 33
6.4.5 Bayes' Theorem 33
6.5 Beta Distribution 34
6.5.1 Experiment Design with Beta Distribution 34

6.6 Adding 35
6.6.1 Sum of Foundational Random Variables 36
6.6.2 Central Limit Theorem 36
6.6.3 Sample Mean and Sample Variance 36
6.7 Bootstraping 37

1 Introduction

1.1 Large Language Models (LLMs)

1.2 Software 3.0

The term "Software 3.0," introduced by Andrej Karpathy, describes how AI is reshaping the way we build software.

- **Software 1.0**: explicit rules written by programmers (if/else logic, loops).
- **Software 2.0**: behavior learned from data (neural networks trained on examples).
- **Software 3.0**: behavior specified in natural language (LLMs following prompts).

1.3 Roles of AI

Over the past decade, AI has moved from a niche technology to a core part of software design. In practice, AI systems tend to play three main roles:

- **AI as a Coding Agent**: AI systems can write, refactor, and review code, accelerating software development. Their output is largely deterministic because it is traditional code that can be inspected, tested, and debugged.
- **AI as a Component**: In this case, AI is embedded within a larger system, and the surrounding architecture is designed around its capabilities and limitations. This is not entirely new: many traditional machine learning models already fit this pattern.
- **AI as a User**: AI systems can also act as users of other systems, because they can consume inputs and generate outputs through your interfaces. Today, system design must account for AI clients that can call APIs, read documentation, and operate on application data. This is introducing a new question: Who are you building for? Human users, AI users, or both?

To design responsibly, we should distinguish when AI is unnecessary, when it is simply convenient, and when it is a true system component. Those choices are important since they respond to the question "What are you building for?" and this mainly depends on how reniewable and fixed we want our outputs.

- **No AI**: Use traditional software when rules are clear, logic is straightforward, and outputs are deterministic and easy to test.

- **AI as a Convenience**: The problem and rules are well understood, but prompting is faster or simpler than writing code. Outputs are still mostly predictable and testable, but reliability checks remain important.
- **AI as a System Component**: Use AI when behavior is too complex or context-dependent to encode with explicit rules. In these cases, conventional testing is harder because expected outputs are open-ended, and the value comes from handling ambiguity (for example, creative generation and open-ended Q&A).

This course focuses primarily on AI as a system component, while still covering the other two cases. In this context, we want to analyze how reviewable and fixed outputs are, and for which kind of users we are building for.

1.4 Challenges

Creating AI systems is more complex than traditional software because of the following challenges:

- **Getting a single call right**: LLMs are not deterministic, and their outputs can vary widely based on the prompt, which needs to be accurately designed to get the desired output.
- **Managing a conversation**: When building a system that interacts with an LLM, you need to manage the context across the conversation, which can make the costs grow rapidly.
- **Long-term memory**: LLMs have limited context windows, so you need to design strategies for storing and retrieving information (like preferences or conversation history) over time.
- **Enterprise/Community knowledge**: LLMs could use private data to train, so we need to design systems that can leverage this knowledge while respecting privacy and security concerns, also of our enterprise.

2 Prompting

Prompting is the process of designing inputs to LLMs in order to obtain the desired outputs. Effective prompting is essential when building AI systems because it directly affects output quality, reliability, and cost.

In this course, we focus on prompting through **API calls** rather than consumer user interfaces. The two settings differ significantly:

- **User Interfaces**: These are designed for end-users and often include features that help guide the user in crafting effective prompts, such as templates, examples, and interactive feedback. They usually don't allow you to access the model directly, so you have less control over the prompting process, but the interface itself can help users get better results.
- **API Calls**: When using APIs, you have more control and flexibility over the prompting process, as you are talking directly to the model. You have to design the prompt yourself, and you can also manage the conversation state and context more effectively. Modern APIs may also expose built-in tools such as web search, code execution, and retrieval, which can further improve results.

In short, API prompting is developer-centric, while user interfaces are user-centric. For instance, obtaining structured output can be difficult in many interfaces but straightforward through an API call.

2.1 Anatomy of a Prompt

A prompt typically consists of the following components:

- **System Prompt**: Sets the persona, rules and constraints the model follows throughout the conversation (job description). Conventionally, the system prompt is the same across all calls, but needs to be repeated since the API is stateless.
- **User Prompt**: The actual task or question you want the model to address. It can include the input data, examples and instructions on how to format the output. Conventionally, the user prompt is the part that changes across calls, depending on the specific task or input data.
- **Parameters**: These are settings that influence the model's behavior, such as temperature, top-p, max tokens, and stop sequences.

2.2 API Parameters

When making API calls to an LLM, you can specify parameters that influence model behavior:

- **Temperature**: Controls output randomness. Higher values (e.g., 0.8) usually produce more diverse and creative text, while lower values (e.g., 0.2) usually produce more focused and consistent text. In practice, the model samples from token probabilities, and temperature shapes that sampling.
- **Top-p**: Controls diversity by sampling only from the smallest set of tokens whose cumulative probability reaches `p`. It is typically used as an alternative to temperature; in many cases, tuning one is enough.
- **Max Tokens**: Caps response length, helping control both cost and verbosity.
- **Stop Sequences**: Define one or more strings that cause generation to stop when encountered.
- **Frequency Penalty**: Penalizes new tokens based on their existing frequency in the text so far, reducing repetition.
- **Presence Penalty**: Penalizes new tokens based on whether they appear in the text so far, encouraging the model to talk about new topics.

Beyond these parameters, other choices also influence output quality, especially **model selection** (for example, different model families) and **API type**.

It is useful to distinguish between **completions** and **responses**: completions are generally **stateless** (each call starts fresh unless you resend context), while responses can be **stateful**, with server-side conversation state that can simplify multi-turn interactions.

2.3 Prompting Strategies

This section covers common prompting strategies that improve model performance. Calibrating specifically is a key part of the design process, as overspecifying or underspecifying the prompt can lead to suboptimal outputs. Cutting redudancy and keeping a good structure permits to get better results, since the model follows literally the instructions

The best strategy to get a good prompt is to iteratively test and refine it, starting with a simple version and gradually adding complexity as needed.

2.3.1 Prompt Structure

Defining a clear **scope** for the model is crucial to prevent it from going off-topic or generating irrelevant information. To better handle complex tasks, it's also important to use **structure formatting** in the prompt, which can help guide the model's attention and improve the quality of the output. An example is to separate instructions, input data and output format into distinct sections, using clear headings or bullet points.
The suggested structure for a prompt is the following:

- **Role (optional)**: Define the role or persona the model should adopt (for example, "You are a helpful assistant"). This differetiates the **direct instruction** prompting strategy, where you simply provide clear, literal instructions for simple tasks, from the **role prompting** strategy, where you shape the tone, vocabulary and depth of the response.
- **Goal**: Clearly state in one sentence the overall goal or task (for example, "Summarize the following text in one sentence"). (This last part doesn't improve factual correctness)
- **Context**: Provide any necessary background information, data, or examples that the model needs to complete the task.
- **Constraints**: Specify any rules like audience, length limits and tone.
- **Output Format**: Define the expected format of the output (for example, "Return a JSON object with the following fields...").
- **Verify**: Ask the model to check its own output for errors or inconsistencies, which can help improve reliability.

Also **where you place instructions** relative to data significantly affects the quality, especially with long data. The common rule is to place long data first, instructions last and query at the very end.

2.3.2 Few-shot prompting

Few-shot prompting involves providing the model with a few examples of the desired output format or behavior within the prompt itself. This can help the model understand the task better and generate more accurate responses.
It's important to choose diverse examples covering typical cases and edge cases, and keep a consistent format across examples to help the model learn the pattern. Also ordering examples matters: place the most similar examples last, closer to the actual task, as the model tends to pay more attention to recent context.

2.3.3 Chain-of-thought prompting

LLMs generate tokens sequentially. When you force intermediate reasoning steps, each subsequent token is conditioned on better context. In other words, the model "thinks" by writing, and by prompting it to write down its reasoning process, you can often improve the quality of the final answer, especially for complex tasks that require multiple steps of reasoning.
There are several variations of this strategy:

- **Zero-shot CoT**: You simply ask the model to explain its reasoning process before giving the final answer, without providing any examples.
- **Few-shot CoT**: You provide examples of the reasoning process along with the final answer, which can help the model learn how to structure its reasoning.
- **Structured CoT**: You explicitly define the intermediate steps or subtasks that the model should follow, which can help guide its reasoning process more effectively.

3 Testing AI Systems

As in the classic software design process, testing is a crucial step in building AI systems, but it also has unique challenges and considerations due to the nature of LLMs and their non-deterministic outputs.
The usual testing process involves the following steps:

- **Define Test Cases**: Identify a set of inputs and expected outputs that cover a range of scenarios, including edge cases and common use cases.
- **Analyze Outputs**: check the failing tests and iterate on the source code or prompt to fix them.

However, when testing AI systems, we need to consider additional factors:

- **Problem Complexity**: usually AI systems are designed to handle complex problems, where no single correct answer exists, and the value comes from handling ambiguity and generating creative solutions. In such cases, defining expected outputs can be challenging;
- **Overfitting to Test Cases**: If you design your prompt to pass specific test cases, you might end up with a prompt that performs well on those cases but fails to generalize to new inputs.

We can think of different approaches to tackle these challenges and improve the testing results of the system. In the following sections, we will explore some of these approaches.

3.1 Single Shot Prompts

The first idea is to use single shot prompts, where the failing test cases are directly included in the prompt as examples of what not to do, or as examples of the desired output format. This can help the model understand the specific requirements and constraints of the task, and improve its performance.

However, this approach can also lead to overfitting if the prompt becomes too tailored to the specific test cases, and it may not generalize well to new inputs, leading to terrible performance when put in production.
A general rule of thumb is to **NEVER include test cases in the prompt** and to never design the prompt to pass specific test cases. The best strategy would be to design the prompt without even knowing what the test cases are.

3.2 Self-Critique

Another approach is to use self-critique, where, in the same prompt, the model is asked to evaluate its own output and identify any errors or inconsistencies. This can help improve the reliability of the output, as the model can catch and correct its own mistakes.

A better approach is to use a separate prompt for self-critique (if possible, also with a different model), where the model is given the original output and asked to evaluate it and to provide a revised version of the output if necessary. Using a separate prompt allows the model to focus solely on the critique process, removing all the previous context and potential biases from the original prompt.

3.3 Feedback Loops

A more advanced approach is to use feedback loops. You can design a system where the model generates an output, then evaluates it against the test cases, and iteratively refines the system prompt to improve performance. There are different strategies to implement feedback loops.

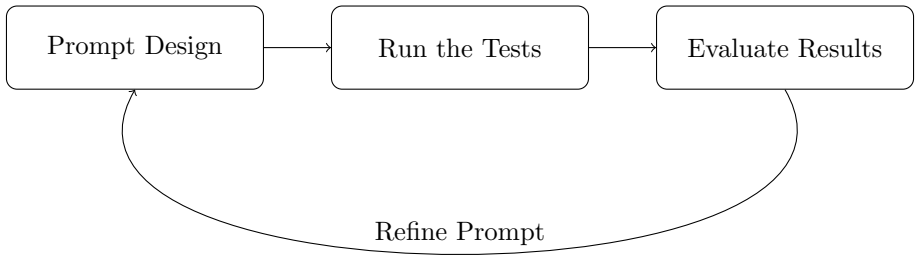

Figure 1: Feedback loop for prompt testing and refinement.

3.3.1 Human in the loop

In the basic implementation of feedback loops, a human is involved in three steps of the process:

- **Prompt Design**: The human designs the initial prompt based on their understanding of the task and the requirements;
- **Evaluate Results**: After executing the prompt on a set of test cases, the human evaluates the results and identifies any errors or inconsistencies;

- **Refine Prompt**: Based on the evaluation, the human refines the prompt to improve its performance.

We can see how this process can be time-consuming and inefficient, especially if the prompt requires multiple iterations to achieve the desired performance. Not only from a time perspective, but also from the knowledge perspective: the human needs to understand the model's behavior and how to design effective prompts, which can be a complex task in itself.

Replacing human evaluation with an LLM judge is the key step to make the loop scalable.

3.3.2 Automatic Feedback Loops

In a fully automated feedback loop, the model itself is responsible for evaluating its own output and refining the prompt accordingly. They are based on the idea of using **LLMs as judges**, where the human is replaced by the model, which is given a set of test cases and asked to evaluate its own output against those cases. It is then asked to provide feedback on how to improve the prompt, and to generate a revised version of the prompt itself, leading to the automatic loop process.

LLM as a Judge Defining a good prompt for the LLM judge is not trivial, and it is crucial to ensure an improved performance of the system. Usually we are interested in defining a score for each parameter we want to evaluate. Based on this, we can also filter out the outputs that do not meet a certain threshold, and manually review them to understand the weaknesses of the system.

Since its cruciality, the model used for this task should be the best available, and it should be different from the one used for the original prompt, to avoid bias and to ensure a more objective evaluation.

Prompt Improvement The prompt improvement step is also crucial, as it is the one that allows the system to learn and improve over time. One of the common approaches is to ask the model to provide many variations of the original prompt (following known prompting strategies), and then to evaluate those variations against the test cases, selecting the best performing one as the new prompt for the next iteration.
Supposing that the Judge is correctly evaluating the outputs, this process can lead to a significant improvement in performance over time.

Wrong approach - Vibe Eval A common mistake is to not implement the Judge, but to have a person testing a few cases manually to check the quality

of the system, and to rely on their "vibe" to decide whether the system is good or not. This approach is obviously not scalable, and it also tends to produce prompts that overfit to the specific test cases.

Avoid Overfitting The automatic approach can still lead to overfitting with small enough test sets, or after a certain number of iterations. It's important, after each iteration, to increase the size of the test set, to ensure that the system is not just learning to pass a small set of cases, but is actually improving its generalization capabilities.

Technical Suggestions In practice, when implementing feedback loops, it is important to consider the following technical suggestions:

- **Use a separate model for the Judge**: This helps to avoid bias and ensures a more objective evaluation of the outputs. The best available model should be used for this task, as the quality of the evaluation is crucial for the improvement of the system.
- **Define clear evaluation criteria**: The prompt for the Judge should include clear criteria for evaluating the outputs. Also, it should be designed as the first step of the process;
- **Batch testing**: Usually testing is done in batches, where a set of test cases is evaluated at once. There are different strategies to implement batch testing:
  - **Trust the Judge**: You can simply ask the Judge to provide a list of elements as output, and then match it with the expected outputs assuming they keep the same order. This is the simplest approach, but even stronger models can make mistakes in this type of tasks, leading to incorrect evaluations.
  - **Use IDs**: A more robust approach is to assign an ID to each test case, and ask the Judge to repeat the ID in the output. This way, we can always map the output back to the original test cases. It's important to also use short IDs, so that the models sees them as a single token.

4 Stateful Agents

The models discussed thus far are **stateless agents**. This means they do not maintain a memory of previous interactions; instead, they treat every request as an independent event and respond based solely on the current input.

To illustrate this, imagine you have access to an infinite sequence of *superhumans*. You approach each one and ask exactly one question. While each superhuman provides a brilliant response to that specific query, they possess no knowledge of your previous conversations with their predecessors. Every time you move to a new person, the context is reset to a blank slate. This behavior characterizes a stateless system.

In contrast, a **stateful agent** retains information across multiple turns. It stores previous inputs and outputs—often referred to as the *conversation history* or *context*—allowing it to understand follow-up questions and maintain continuity in a dialogue.

In this section we will explore various techniques used in agents design, from simple stateless agents to more complex stateful ones.

4.1 Stateless

As mentioned, stateless agents do not maintain any memory of past interactions. The `chat completition` api is an example of a stateless agent by design.

```
# Turn 1: tell the model your name
r1 = client.chat.completions.create(
  model="gpt-4.1-mini",
  messages=[{"role": "user", "content": "My name is Alice."}],
)
# Turn 1: "Nice to meet you, Alice!"

# Turn 2: ask for your name  but we DON'T send turn 1
r2 = client.chat.completions.create(
  model="gpt-4.1-mini",
  messages=[{"role": "user", "content": "What is my name?"}],
)
# Turn 2: "I don't have access to your name."
```

The model can't answer because it literally never saw the first message.

Although this type of agents seems pretty useless, they can be fine for single-shot tasks, like writing a poem, generating an image, or answering a question.

4.2 Default Stateful Agents

Open AI `Response API` can be used to create stateful agents.

```
resp1 = client.responses.create(model="gpt-4.1-mini", input=
    "My name is Alice.")

resp2 = client.responses.create(
    model="gpt-4.1-mini",
    input="What is my name?",
    previous_response_id=resp1.id,    # server remembers turn
        1
)
# "Your name is Alice."
```

This is convenient, but it's a black box approach. We have no control over what the model remembers and how it uses that information.

All the next techniques we will manage the state ourselves, giving us more control and flexibility.

4.3 Full History

A first approach to create a stateful agent is to send with each request the full conversation history. This is obviously the most straightforward way to maintain context, but it can quickly become inefficient as the conversation grows longer. In fact, each time we send a new message, the number of tokens we need to send increases quadratically, since we need to resend all previous messages.

This is not only inefficient in terms of latency and cost, but it can also lead to fulling the model's context window, which can cause lower quality responses or even errors.

4.4 Sliding Window

To mitigate the issues of the full history approach, we can use a **sliding window** technique. Instead of sending the entire conversation history, we only send the most recent N messages, where N is a predefined window size. This way, we can maintain some context while keeping the number of tokens manageable and constant over time.

The downside of this approach is that we might lose important information from earlier in the conversation, which can lead to less coherent responses if the model relies on that information.

4.5 Summarization

To address the limitations of the sliding window technique, we have to find a way to retain important information from the conversation without sending the entire history. This is where **summarization** comes into play. The idea is to periodically summarize the conversation history and send the summary along with the most recent messages. This way, we can keep the context manageable while still retaining important information from earlier in the conversation.

The summarization is usually done by the model itself. Along with the user message, we can provide the model with the current summary and ask it to update the summary based on the new message.

This way, the memory stays bounded, without the loss of information.

A few tips for effective summarization prompts:

- Use a structured format for the summary, organizing information into categories (e.g., "Facts", "Opinions", "Actions", etc.);
- Set a maximum length for the summary to ensure it remains concise and focused on the most relevant information;
- Resolve contradictions in the summary by asking the model to identify and address any conflicting information.

4.6 Long-Term Memory

Every technique we have seen so far is based on the idea of a *working memory*, that dies whenever the conversation ends. However, in many applications we want to have a *long-term memory* that persists across multiple conversations and sessions.

This is usually achieved by:

- Once the conversation ends, extract important information. Ask the LLM to pull out structured facts worth remembering: facts, preferences, decisions, expertise, etc;
- Store this information in an external database;
- When a new conversation starts, retrieve relevant information from the database and inject it into the prompt.

This is the actual pattern used behind the OpenAI `ChatGPT` memory feature.

4.7 When to use which pattern

Each strategy can be used in different scenarios, depending on the specific requirements of the application and the constraints of the model being used.

- **Stateless**: suitable for single shot tasks (e.g., translate, classify);
- **Full History**: suitable for short conversations;
- **Sliding Window**: suitable for longer conversations where only the most recent context is relevant;
- **Summarization**: suitable for long conversations where important information needs to be retained without exceeding the context window;
- **Long-Term Memory**: suitable for applications that require personalization or have returning users, where information needs to persist across sessions.

5 Agent Tools

LLMs by themselves are very powerful when it comes to understanding and generating human language. However, they can be even more powerful when combined with external tools that allow them to perform specific tasks or access up-to-date information. In this section, we will explore some of the tools that can be integrated with LLMs to enhance their capabilities.

They enable a wide range of applications, such as:

- **Computational tools:** perform complex calculations, data analysis, and other computational tasks that may be beyond their native capabilities;
- **Knowledge retrieval tools:** access external databases, APIs, or the internet to retrieve up-to-date information;
- **Action execution tools:** interact with other software or hardware to perform actions based on the LLMs output.

5.1 Register a Tool

LLM providers usually offers ready-to-use tools, such as a web search tool, code interpreter, file search, and more. However, they also allow users to create and register custom tools that can be tailored to specific use cases or domains.

To register a tool, we first need to define its schema, which includes the tool's name, description, and input parameters. An example of a tool schema is shown below:

```
{
"type": "function",
"function": {
    "name": "explore_data",
    "description": "Inspect dataset schema, missing values, and candidate targets. Call first when the dataset structure is unknown.",
    "parameters": {
    "properties": {
        "sample_rows": {"type": "integer", "description": "Number of sample rows to return"}
    },
    "required": ["sample_rows"]
    }
}
}
```

5.2 Multi-Tool loop

Real world problems often require the use of multiple tools in a sequence, where the output of one tool serves as the input for another. The model must reason about which tool to call next based on what it's learned so far.

Example: Data Analysis Agent Consider the following example, of a data analysis agent. The following tools are registered:

- **Explore Data:** inspects the dataset schema, missing values, and candidate targets. it has no preconditions;
- **Cluster Data:** performs clustering on the dataset, discovering natural groupings in the data. Needs a numeric column as input;
- **Classify Data:** performs regression or classification on the dataset, depending on the target variable. Needs a target column as input.

Different queries may require different sequences of tools. For example:

- **Query 1:** "What kind of data do I have?" → Explore Data;
- **Query 2:** "Are there any natural customer segments in the data?" → Explore Data → Cluster Data;
- **Query 3:** "Segments and churn prediction for age=29, income=50k..." → Explore Data → Cluster Data → Classify Data.

5.3 Problems with tool use

The usual problem with multi-tool agents is that they can get stuck in loops, calling the same tool repeatedly without making progress towards the final goal. To mitigate this issue, we can implement a control mechanism that allows the use of a tool if and only if the following conditions are met:

1. Some part of the original query is still unanswered;
2. Another available tool can resolve it;
3. The tool call is necessary, not just helpful.

This should help the agent avoid common pitfalls, such as calling tools just for the sake of calling them, or getting stuck in a loop with no progress.

Another approach, particularly useful when queries are complex or ambiguous, is to allow the agent to ask for clarification from the user before deciding which tool to call next.

Keep in mind that how we design the tools and their interfaces can also influence how effectively the agent uses them. In fact, the model picks them based on their descriptions and parameters. Therefore, it's important to make them as clear and informative as possible, to help the model understand when and how to use them effectively.

6 Evaluation

In this section, we will discuss the basic recipe for "evaluating" in different periods of software development, starting from the early days of software development, to the current era of Software 2.0 and 3.0, focusing on the differences in the evaluation process and the challenges that arise in each era.

6.1 Software 1.0 - Evaluation in Traditional Software Development

In traditional software development, the evaluation process is relatively straightforward. The expected output of the software is well-defined, and we can easily compare the actual output with the expected output to determine if the software is working correctly. This process is often referred to as "testing", and it can be done through:

- **Unit Testing:** testing individual components or functions of the software to ensure they work as expected. For example, if we have a function that adds two numbers, we can test it by providing different pairs of numbers and checking if the output is correct;
- **Integration Testing:** testing the interaction between different components of the software to ensure they work together correctly;
- **Regression Testing:** testing the software to ensure new changes do not break existing functionality.

The process is **deterministic** and **binary**, meaning that the software either works or it doesn't, and the output is always the same for the same input. The only difficulty in this process comes from the coverage of the tests, not from the evaluation itself.

6.2 Software 2.0 - Evaluation in Machine Learning

In the era of Software 2.0, where machine learning models are used to perform tasks, the evaluation process becomes more complex. This is because the output of the model is **not deterministic** anymore, but rather **probabilistic**, The evaluation process, also called **experimentation**, is no longer binary, but rather a matter of degree. It shifts from "is the output correct?" to "how good is the model across a test set?".

For our analysis, we will focus on the evaluation of a binary classification model, where the goal is to classify inputs into two categories (for example, "spam" vs "not spam", or "positive" vs "negative"). Usually the metrics used is the proportion of misclassified inputs (also known as **error**), meanwhile the proportion of correctly classified inputs is called **accuracy**.

6.2.1 Comparing Models

Before putting a model into production, we want to compare it with other models to understand which one performs better on a given task. We could also compare different versions of the same model, to understand if the changes we made have improved the performance or not.

In simple cases, such as in 2D dataset, we can plot the predicted output and visually inspect the decision boundary to answer these questions. However, in more complex cases, such as in high-dimensional datasets, we need to rely on quantitative metrics to compare the models.

The most straightforward approach is to train both models on the same training set, and pick the one with fewer mistakes. This generates a few questions:

- On which data we compute the error?
- How we eliminate random effects?
- Are the metrics we are using good?

6.2.2 Data Splitting

As you have surely heard before, you should **never judge your performance on the training set**, since the model has already seen them and can easily memorize them, leading to an overestimation of the model's performance on unseen data.

Test Set

The first thing that should be always done in machine learning is to split the original dataset into a **training set** and a **test set**. The training set is used to train the model, while the test set is used to evaluate the model's performance on unseen data, which is more representative of the real-world performance of the model.

How should we split the dataset? The only factor to consider when splitting the dataset is the **size** of the **test set**. The bigger it is, the more it is representative of the real-world data distribution, and the more reliable the evaluation results will be.

Ideally, we want 10 000 examples in the test set, but this is not always possible, especially if the original dataset is small.

Overfitting the test set

However, this approach can lead to overfitting on some cases. Suppose we want to choose the best hyperparameters for our model, and we are provided with a test set. We try different hyperparameter settings, and we pick the one that performs best on the test set, supposing that it will also generalize well to unseen data.

At a certain point, we are provided with a fresh test set, and we find out that the performance is much worse than expected with the chosen hyperparameters. This is because we have **overfitted the test set**, meaning that we have chosen hyperparameters that perform well on the test set, but not on unseen data. Our method for choosing the best hyperparameters are another learning algorithm, and by testing different hyperparameters on a small test set, we are essentially training this learning algorithm on the test set, leading to overfitting.

We can see this as a form of `multiple testing` problem in statistics. We are testing multiple hypotheses (different hyperparameter settings) on the same test set, and we are picking the one that performs best, which is likely to be a false positive.

The simple answer to this problem is to **never use the same test set more than once**, and to always use a fresh test set for the final evaluation of the model's performance.

Validation Set

Reusing the test set not only makes you choose the wrong model, but also **inflates the performance measures**, making them look better than they actually are.

To avoid this, we need to choose the best best model and/or hyperparameters on a separate set of data, extracted from the training set and called the **validation set**. Ideally, the validation set is the same size of your test set, but you can make it a little smaller to get more data for training.

Note that this approach alone does not solve the problem of multiple testing, but it just provides a final fail-safe to detect it, You need always to be careful and whenever it's needed to rerun your experiments on training and validation sets, as you can easily end up overfitting the validation set as well.

However, if your dataset is large, and you haven't done anything strange in tuning phase, the validation set results should be a good estimate of the test set results, and you can be reasonably confident that the model will perform well on unseen data.

Cross Validation

In some cases, especially when the dataset is small, we can't afford to set aside a large portion of the data for testing and validation, since we need as much data as possible for training.

A technique that can help in this case is called **Cross validation**. It allows us to use the entire dataset for both training and testing, by splitting it into multiple folds. For example, in a 5-fold cross validation, we split the dataset into 5 equal parts, and we train the model on 4 parts and test it on the remaining part (validation). We repeat this process 5 times, each time using a different part as the test set,and we average the results to get a more reliable estimate of the model's performance.

This can be time-consuming, but ensures that every instance in the dataset is used for training. After chosen your model and hyperparameters through the cross-validation process, you can test it once on the test set to get an estimate of its performance on unseen data.

A special case: Temporal Data

If your data has a temporal ordering, you need to take it into account when splitting the dataset. In such cases, you can't sample your test set randomly, since training on future data and testing on past data is not a realistic scenario. The ordering in the dataset needs to be maintained when splitting the dataset, to ensure that the model is trained on past data and tested on future data, which is more representative of the real-world scenario.

However, this is not always the case. If your data has a timestamp, but there's no meaning associated with the ordering of the data (such as in email classification), you can still split the dataset randomly, since the ordering does not provide any useful information for the task at hand.

If you want to apply cross validation to temporal data, that takes a small portion of initial data as the training set and then iteratively adds more data to the training set, while validating on the next portion of data, until you reach the end of the dataset.

Final remark: Philosophically talking, when in doubt make sure that the evaluation setting accurately simulates production, and that the validation setting accurately simulates the evaluation setting,

6.2.3 Experiment Design

Now that we now how to experiment, what experiments should we run? Which hyperparameters should we test? Basically as long as you are not looking at the

test set, you can do whatever to like.

Hyperparameter Tuning

When tuning hyperparameters, we have the following options:

- **Trial and Error**: This is the most basic approach, where you simply try different combinations of hyperparameters and see which one performs best on the validation set. This is driven by intuition and experience, and it can be time-consuming, especially if you have a large number of hyperparameters to tune.
- **Grid Search**: This is an automated approach, where you define a grid of relevant hyperparameter values, and the algorithm trains and evaluates a model for each combination of hyperparameters in the grid. This can be computationally expensive, especially if the grid is large, but it ensures that you are exploring a wide range of hyperparameter combinations.
- **Random Search**: This is another automated approach, where you randomly sample combinations of hyperparameters from a defined distribution, and train and evaluate a model for each combination. This can be more efficient than grid search, especially if you have a large number of hyperparameters, since it allows you to explore a wider range of combinations.

6.3 Software 2.0 - Statistical Testing

Machine Learning and statistics are closely related, but significance testing is rarely used in ML experiments due to large sample sizes and emphasis on replication over statistical significance. Replication is key, but often challenging in practice, so we can still use statistical testing as a tool to assess the reliability of our results.

6.3.1 True metrics vs Estimated metrics

When we evaluate a model, we usually imagine data being sampled from a probability distribution $p(x)$. What we really want is to develop a classifier that performs well on any data point sampled from the original distribution.

Imagine for example we sample one data point from the distribution, and we classify it with some classifier c. The probability of c making a correct prediction on this data point is the **true accuracy**. However, we can't compute the true accuracy, since we don't have access to the entire distribution, but only to a finite sample of data points.

Instead, what we usually do is to compute the so-called **sample accuracy**. We take a large enough sample of data points from the distribution, and we compute the accuracy of the classifier on this sample, which is an estimate of the true accuracy.

Now, given the estimated sample accuracy, the question is: how sure can we be that the true accuracy is close to the sample accuracy?

Confidence Intervals

One way to quantify the uncertainty around our estimate is to compute confidence intervals, which are ranges of values that are likely to contain the true metric with a certain level of confidence. This helps us understand the uncertainty around our estimate and make more informed decisions about model performance.

The confidence interval is computed based on the variability of the sample metric across different samples, and it gives us a range of values within which we can be reasonably confident that the true metric lies.

The size of the confidence interval depends on two factors:

- **True metric**: If the true metric is close to 0 or 1, the confidence interval will be narrower, since there is less variability in the sample metric.
- **Sample size**: The larger the sample size, the narrower the confidence interval, since there is less variability in the sample metric across different samples.

If you have not the luxury of a large sample size, you can do some statistical testing to understand how much the sample metric is close to the true metric.

Remark: Since we are working with estimated values, the confidence interval itself is an estimate. It is not guaranteed to contain the true metric with the specified confidence level for any single realization. The interval is relative to the specific test sample collected; therefore, it is not a fixed range but one that varies across different samples.

Standard Deviation and Standard Error

The **standard deviation** (σ) measures the amount of variation or dispersion within a set of data values. In the context of a sampling distribution, the region spanning approximately two standard errors to the left and two to the right of the sample mean (a total range of four standard errors) contains roughly 95% of the sample metrics across different samples. Consequently, this interval

$[\bar{x}-2SE, \bar{x}+2SE]$ represents a 95% **confidence interval** for the true population metric.

The **standard error** (SE) is the standard deviation of the sampling distribution of a statistic, most commonly the sample mean. It quantifies the uncertainty of the sample estimate. It is computed as:

$$SEM = \frac{\sigma}{\sqrt{n}}$$

Where σ is the standard deviation of the individual observations and n is the sample size.

While the standard deviation is a measure of the **spread** of the underlying data, the standard error and the confidence interval are measures of **uncertainty** regarding the true population parameter. Increasing the sample size n will typically decrease the standard error, thereby narrowing the confidence interval, even if the underlying standard deviation of the population remains constant.

6.3.2 Randomness

In machine learning, the results of an experiment can vary significantly upon rerunning due to **random initialization** of model weights and stochastic optimization processes. Additionally, **data sampling** introduces randomness because the available dataset is merely a single realization of a larger, underlying population.

To evaluate robustness to random initialization, executing the experiment multiple times with different seeds is often sufficient.

Instead, to assess how much our evaluation metric might fluctuate due to data sampling, we rely on **resampling methods**. The core idea is to repeatedly recompute the sample metric on different subsets of the data, allowing us to estimate its standard deviation and construct confidence intervals for the true metric.

Resampling Methods

In cross-validation, we compute the metric on each splitted fold. By analyzing how much the metric varies across these folds, we gain insight into the robustness of our results and the degree to which they depend on the particular sample at hand.

However, as we increase the number of repetitions, especially with small datasets, the overlap between samples also increases, which can limit the independence of our estimates.

Bootstraping Methods

An alternative approach is to use **bootstrapping methods**, where we create new datasets by **sampling with replacement** from the original dataset. Each **bootstrap sample** is the same size as the original dataset, but because sampling is done with replacement, some data points may appear multiple times or not at all.

This technique allows us to create as many resampled datasets as needed, without being limited by overlap concerns. By evaluating our metric on each bootstrap sample, we can estimate its variability and construct confidence intervals, often with greater flexibility than traditional resampling methods.

In practice, bootstrapping provides a powerful way to approximate the underlying data distribution.

6.3.3 Evaluation Metrics - Regression

When evaluating regression models, we need to compute continuous metrics such as the **mean squared error** (MSE). Here, we assume that each observation is drawn from a distribution centered around some true underlying value.

The sample mean, calculated from our test results, provides a point estimate of this true metric. However, it is important to remember that the sample mean itself is a random variable, and it will vary from sample to sample.

For small sample sizes, the distribution of the sample mean follows a Student's t-distribution, which approaches a normal distribution as the sample size increases ($n > 30$), both centered at the true mean. This statistical framework enables us to construct confidence intervals for the true metric, using the sample mean and its observed variability across samples.

Other metrics for regression include the **mean absolute error** (MAE), which measures the average absolute difference between the predicted values and the true values, and the **root mean squared error** (RMSE), which is the square root of the MSE and provides an interpretable metric in the same units as the original data.

Bias and Variance

Bias and variance are two fundamental concepts to keep in mind when evaluating regression models.

- **Bias**: This is the difference between the **optimal MSE** (the lowest possible error for your model class) and the true MSE. Bias reflects how much your model is systematically underfitting the data. Importantly, bias remains unchanged even if you resample the data, as it is an inherent property of the model itself, not the data.

- **Variance**: This is the difference between the true MSE and the observed MSE across different samples. Variance captures how much your model's predictions fluctuate due to variations in the training data. Unlike bias, variance can change from sample to sample, since it depends on the data rather than the model.

Typically, there is a trade-off between bias and variance: increasing model complexity tends to reduce bias but increase variance, while simpler models have higher bias and lower variance.

6.3.4 Evaluation Metrics - Classification

When evaluating classification models, we rely on metrics such as accuracy, precision, recall, and F1-score. These metrics are derived from the confusion matrix, which summarizes the counts of true positives, true negatives, false positives, and false negatives.

Confusion Matrix

The confusion matrix summarizes a classification model's performance by counting true positives, true negatives, false positives, and false negatives. From this table, we can compute a variety of metrics to evaluate the model, for example the **accuracy**, which is the proportion of correctly classified instances over the total number of instances. While it can be challenging to directly compare two models using only their confusion matrices, this tool provides valuable insights into the types of errors a single model makes.

Application of Confusion Matrix

Plotting the confusion matrix for the training, validation, and test sets provides valuable insights into model behavior and helps diagnose issues like overfitting or poor generalization.

Typically, we use **test accuracy** to assess the validity of our conclusions, **validation accuracy** to tune hyperparameters, and **training accuracy** to compare against validation accuracy (the **generalization gap**) to detect overfitting or underfitting. Training accuracy alone should never be used to evaluate a model, as it does not reflect how well the model will perform on unseen data.

Imbalance Issues

Imbalance issues arise when classes are not equally represented in the dataset or when the cost of misclassifying one class is much higher than the other.

Class Imbalance occurs when one class is much more frequent than the other, making high accuracy misleading. For example, a model may simply predict the majority class and still achieve a high accuracy, while failing to correctly identify instances of the minority class. If the minority class is underrepresented in the training set, the model may fail to learn about it, resulting in poor generalization to new data.

For example, consider a machine learning model designed to detect a rare disease, where only 1% of the cases are positive. If someone comes up with a model with a 99% accuracy it might seem impressive at first glance, but in reality, it could be a model that simply predicts "negative" for every case, thus failing to identify any of the actual positive cases.

So, every time someone tells you that their model has a high accuracy, you should always ask about the class distribution in the dataset.

Cost Imbalance the cost of misclassifying one class can be much higher than the other (e.g., failing to detect a sick patient is typically worse than a false alarm). In such cases, it is important to use metrics and error calculations that reflect both class imbalance and cost imbalance, often by weighting errors according to their impact based on the specific application.

Considering the same example of a disease detection model, if the cost of missing a positive case (false negative) is much higher than the cost of a false alarm (false positive), we might want to prioritize the model's ability to correctly identify positive cases, even if it means accepting a higher number of false positives.

Precision and Recall

Precision measures the proportion of predicted positives that are actually correct, while **recall** measures the proportion of actual positives that are correctly identified.

$$Precision = \frac{TP}{TP + FP} \qquad Recall = \frac{TP}{TP + FN}$$

There is often a trade-off between precision and recall, especially in the presence of class imbalance: increasing one can decrease the other. Plotting precision and recall for different classifiers or thresholds helps visualize and manage this trade-off.

ROC Space: True Positive Rate vs False Positive Rate

TPR (True Positive Rate) measures the proportion of actual positives correctly identified, while **FPR** (False Positive Rate) measures the proportion of actual negatives incorrectly classified as positives.

$$TPR = \frac{TP}{TP + FN} \qquad FPR = \frac{FP}{FP + TN}$$

The trade-off between maximizing TPR and minimizing FPR is visualized in the **ROC curve** (Receiver Operating Characteristic), where the ideal point is TPR = 1 and FPR = 0. The ROC curve helps compare classifiers across different thresholds.

Plotting thresholds

So far, we have discussed comparing different models, but we can also evaluate a single model by varying its decision threshold. This analysis is meaningful and possible only if the model is a **ranking classifier**, that is, it outputs a continuous score or probability for each instance. For ranking classifiers, **ranking error** refers to cases where the model assigns high confidence to incorrect predictions (e.g., predicting positive with high confidence when the true label is negative).

For instance, when using a high threshold ("shy" model) results in few positive predictions, while a low threshold ("bold" model) predicts more positives, increasing the risk of false positives. Plotting performance metrics (such as ROC or Precision-Recall curves) across thresholds helps visualize this trade-off and select the optimal threshold for a specific application.

Coverage Matrix

To also consider how sure our model is about its predictions, we can use a **coverage matrix**. We first need to turn our regular classifier into a **ranking classifier**, by using the confidence scores as the output of the model, instead of the binary predictions. Such models cannot be evaluated on the test data, since we don't know what the correct ranking is.

For ranking such classifiers, a **coverage matrix** visualizes all possible pairs of negative (columns) and positive (rows) examples. Each cell compares a positive example x^+ and a negative example x^- using the model score $s(\cdot)$:

- **Green cell**: $s(x^+) > s(x^-)$ — the model correctly ranks the positive above the negative.
- **Red cell**: $s(x^+) \leq s(x^-)$ — the model ranks the pair incorrectly (ranking error).

The proportion of red cells is the probability of making a ranking error.

By introducing a threshold τ on $s(\cdot)$, we turn the scores into binary predictions: $s(x) \geq \tau$ is classified as positive. Varying τ changes which pairs are correctly ordered, linking the coverage matrix to the ROC curve.

Area Under the Curve (AUC)

The **AUC** (Area Under the Curve) is a single metric that summarizes a model's performance across all possible thresholds. It is calculated as the area under the ROC curve and is widely used to compare different classifiers.

Intuitively, the AUC represents the probability that a ranking classifier assigns a higher score to a randomly chosen positive example than to a randomly chosen negative one. In other words, it quantifies how well the model ranks positive examples above negative ones, regardless of the specific threshold used for classification. In terms of the coverage matrix, the AUC is the proportion of green cells (correctly ranked pairs).

Partitioning Classifiers For some models, such as decision trees (**partitioning classifiers**), scores are assigned based on regions or leaves. If we use the proportion of positive points in each leaf as the score, all points in the same leaf share the same score. This can result in ties, where some pairs cannot be strictly ranked. For large datasets, these ties have less impact on the overall AUC.

Precision-Recall Curve

The **Precision-Recall (PR) curve** is another tool for evaluating classification models. Instead of plotting TPR vs. FPR (as in the ROC curve), the PR curve plots precision against recall for different thresholds.

The area under the PR curve (AUC-PR) provides a single-number summary, which is especially informative when dealing with imbalanced datasets—situations where the ROC curve may be overly optimistic.

However, the PR curve is not normalized and can be harder to interpret than the ROC curve, as its baseline depends on the proportion of positive examples. Both curves offer complementary insights, so it is often helpful to plot and compare them together.

6.4 No Free Lunch Theorem

It is natural to ask: which classifier, model, or search method is the best? The **No Free Lunch Theorem** tells us that, without knowing the specific problem, there is no universally best algorithm.

If we average over all possible problems, every optimization or learning algorithm performs equally well. An algorithm only excels when its assumptions match the problem at hand. This is known as the **problem of induction**. Every learning algorithm or model has an **inductive bias**: a set of built-in assumptions about the data. Inductive bias determines how a model generalizes from training

data to unseen data. For example, a linear model assumes linear relationships between features and the target variable.

As practitioners, our task is to choose the inductive bias that best fits our problem. When two models explain the data equally well, we usually prefer the simpler one, following the principle that simple patterns are more likely than complex ones. This is why many learning methods include a built-in **bias toward simplicity**.

7 Experiment Design

7.1 Introduction

Any measurement, without knowledge of the uncertainty, is **meaningless**. Here, "meaningless" means a measurement lacks reliability if its uncertainty is unknown; without knowing its precision or accuracy, the value cannot be trusted or interpreted and may be **misleading**.

Experimental measurements are essential to understand system behavior, allowing us to:

- Decide to **ship or not**
- **Identify** improvements for next iterations
- **Communicate** system behavior to others

This enables estimation of system performance for properties and conditions of interest.

7.2 Basic, Probabilities and Random Variables

7.2.1 Definition of Probability

From a general perspective, probability is a number between 0 and 1 to which we assign a meaning. A frequentist defines it as "*A proportion (# of times an event occurs)/repetitions as experiments repetitions tends to infinity*". However, this approach cannot work with us, since we cannot repeat experiments infinitely, and we cannot experiment always in the same conditions.

So, more generally the definition on which we are interested is "*A belief/likelihood of an event occurring*" or "*a belief on the distribution of a random variable*".

More formally we define:

- **Sample Space (S)**: The set of all possible outcomes of an experiment.
- **Event (E)**: An event is a subset of the sample space that satisfies some conditions.
- **Probability (P(E))**: A measure of the likelihood of an event occurring, defined on the sample space.

More formally, we can define the two interpretations of probability as follows:

- **Frequentist Interpretation**:
$$P(E) = \lim_{n\to\infty} n(E)/n$$
where $n(E)$ is the number of times event E occurs in n trials.
- **Bayesian Interpretation**:
$$P(E) = P(E|I) = belief$$
where I is the background information and belief is a subjective degree of confidence in the occurrence of event E. This can be seen as an estimation of the frequentist probability, where the belief is updated as new evidence is obtained, following Bayes' theorem.

7.2.2 Axioms of Probability

For both interpretations, we have the following axioms of probability:

- **Non-negativity**: $P(E) \geq 0 \forall E \in S$.
- **Normalization**: $P(S) = 1$.
- **Complement Rule**: $P(E') = 1 - P(E)$, where E' is the complement of E.

Then if an event E is **mutually exclusive** with another event F (i.e., they cannot occur simultaneously), we have:
$$P(E \cup F) = P(E) + P(F) \tag{1}$$
This is the simplest case, and can be extended to more than two events.

In the case we have all outcomes **equally likely**, we can calculate the probability of an event E as:
$$P(E) = \frac{|E|}{|S|} \tag{2}$$
And works only if we have a finite sample space, and all outcomes are equally likely.

7.2.3 Random Variables

We define a **random variable** as a function that maps each outcome of an experiment to a real number, capturing uncertainty. Assigning a value to a random variable defines an event, and we can describe its behavior with a probability distribution.

For example, in a coin toss, let X be 1 for heads and 0 for tails. This is a **binary random variable** since it takes two values. If both outcomes are equally likely, the probability distribution is:
$$P(X = 1) = 0.5, \quad P(X = 0) = 0.5 \tag{3}$$

Probability Mass Function (PMF) The **probability mass function (PMF)** of a discrete random variable gives the probability that it equals a specific value: $P(X = k)$ for each possible k. The PMF represents the distribution and allows calculation of event probabilities.

7.3 Foundational Random Variables

Some random variables are classic and foundational in probability theory. They can be used to model various phenomena and are essential for understanding uncertainty in experiments.

7.3.1 Binomial Random Variable

The scenario we are describing is the following:

- n independent trials
- A consistent probability of success p in each trial, binary outcomes (success/failure)
- We want the probability of observing k successes in those n trials

The probability of observing k successes in n trials is given by the **binomial distribution**:

$$X \sim Bin(n, p) \tag{4}$$

Where X is the random variable representing the number of successes, and its PMF is:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k} \tag{5}$$

Where $\binom{n}{k}$ is the binomial coefficient, calculated as:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \tag{6}$$

The binomial distribution is useful for modeling scenarios like coin tosses.

If we plot the PMF of a binomial distribution with $n = 10$ and $p = 0.5$, we would see a bell-shaped curve centered around $k = 5$, indicating that observing 5 successes is the most likely outcome.

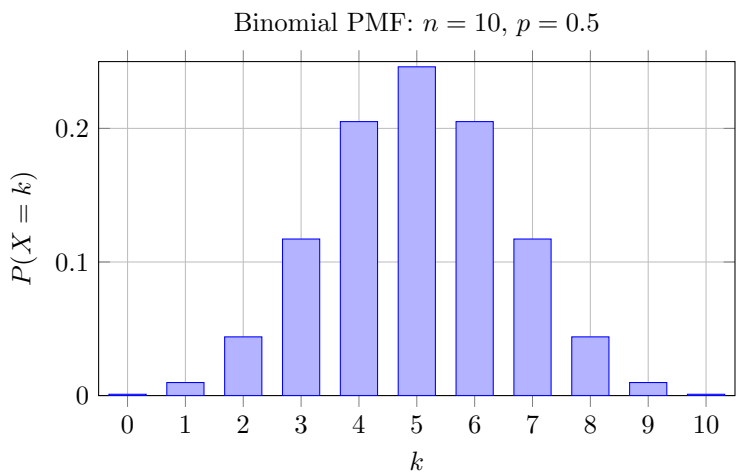

Figure 2: PMF of the binomial distribution with $n = 10$, $p = 0.5$

7.3.2 Bernoulli Random Variable

The Bernoulli distribution is a special case of the binomial distribution where $n = 1$. It models a single trial with two possible outcomes: success (1) with probability p and failure (0) with probability $1 - p$. The PMF of a Bernoulli random variable X is:

$$P(X = 1) = p, \quad P(X = 0) = 1 - p \tag{7}$$

7.3.3 Other PMFs Random Variables

Other important discrete random variables include:

- **Geometric**: Models the number of trials until the first success in a sequence of independent Bernoulli trials.

$$P(X = k) = (1 - p)^{k-1} p$$

- **Negative Binomial**: Models the number of trials until a specified number of successes occurs.

$$P(X = k) = \binom{k-1}{r-1} p^r (1 - p)^{k-r}$$

7.4 Mastering Uncertainty

7.4.1 Conditional Probability

The **conditional probability** of an event E given another event F occurred is defined as:

$$P(E|F) = \frac{P(E \cap F)}{P(F)} \tag{8}$$

The **chain rule** allows us to express the joint probability of multiple events in terms of conditional probabilities:

$$P(E \cap F) = P(E, F) = P(E|F)P(F) = P(F|E)P(E) \tag{9}$$

The **generalized chain rule** allows us to express the joint probability of multiple events in terms of conditional probabilities:

$$P(E_1, E_2, ..., E_n) = P(E_1)P(E_2|E_1)P(E_3|E_1, E_2)...P(E_n|E_1, E_2, ..., E_{n-1}) \tag{10}$$

If we have $P(F) = 0$, then $P(E|F)$ is undefined, in other words, we cannot condition on an event that has zero probability of occurring.

If E and F are **independent** events, then the occurrence of one does not affect the probability of the other, and we have:

$$P(E \cap F) == P(E|F)P(F) = P(E)P(F) \tag{11}$$

7.4.2 Expectation

The **expectation** is the summation of all possible values of a random variable, weighted by their probabilities. It's also called mean, weighted average, or center of mass, and represents the average value we would expect from a random variable.

$$E[X] = \sum_{x \in S} xP(X = x) \tag{12}$$

If we want to calculate the calculation of the expectation from data, we can use the following formula:

$$E[X] \approx \frac{1}{n} \sum_{i=1}^{n} x_i \tag{13}$$

Where x_i are the observed values of the random variable in our data, and n is the number of observations. Basically, we are calculating the average of the observed values, which is an estimate of the true expectation.

The expectation has several important **properties**:

- **Linearity**:
$$E[aX + bY] = aE[X] + bE[Y]$$
- **Expectation of a sum**:
$$E[X + Y] = E[X] + E[Y]$$
- **Unconscious Statistician**:
$$E[g(X)] = \sum_{x \in S} g(x)P(X = x)$$
- **Law of Total Expectation**: For any random variables X and Y, we have:
$$E[X] = \sum_{y \in S} E[X|Y = y]P(Y = y)$$
Where $E[X|Y = y]$ is the conditional expectation of X given $Y = y$.

Foundational random variables have specific expectations:

- **Binomial random variable** $X \sim Bin(n, p)$, the expectation is:
$$E[X] = np$$
- **Bernoulli random variable** $X \sim Ber(p)$, the expectation is:
$$E[X] = p$$
- **Geometric random variable** $X \sim Geo(p)$, the expectation is:
$$E[X] = \frac{1}{p}$$

However, the expectation is just a summary statistic, and it does not capture the variability of the random variable. If not used carefully, it can be misleading, since it does not provide information about the spread or shape of the distribution.

7.4.3 Variance

The **variance** of a random variable measures the spread or variability of its distribution. It is defined as the expected value of the squared deviation from the mean:
$$Var(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2 \tag{14}$$
The variance is expressed in the square of the units of the random variable, which can make it less interpretable. Then can be heavily influenced by outliers, as it squares the deviations from the mean.

Key properties of variance include:

- **Non-negativity**: for any random variable X

$$Var(X) \geq 0$$

- **Additivity for Independent Variables**: If X and Y are independent random variables

$$Var(X+Y) = Var(X) + Var(Y)$$

.

- **Scaling**:for any constant a

$$Var(aX) = a^2 Var(X)$$

- **Shift Invariance**: for any constant c

$$Var(X+c) = Var(X)$$

- **Variance of a Sum**: For any random variables X and Y

$$Var(X+Y) = Var(X) + Var(Y) + 2Cov(X,Y)$$

- **Variance of Conditional Probability**: For any random variables X and Y

$$Var(X) = E[Var(X|Y)] + Var(E[X|Y])$$

7.4.4 Law of Total Probability

The **law of total probability** states that if $E_1, E_2, ..., E_n$ are mutually exclusive and exhaustive events, then for any event F, we have:

$$P(F) = \sum_{i=1}^{n} P(F|E_i)P(E_i) \tag{15}$$

This law allows us to calculate the probability of an event F by considering all possible ways that F can occur through the events E_i. If we have only two events E and E^C, then the law of total probability simplifies to:

$$P(F) = P(F|E)P(E) + P(F|E^C)P(E^C) \tag{16}$$

7.4.5 Bayes' Theorem

Bayes' theorem allows us to update our beliefs about the probability of an event E given new evidence F:

$$P(E|F) = \frac{P(F|E)P(E)}{P(F)} \tag{17}$$

All the terms in Bayes' theorem have specific names:

- $P(E|F)$ is the **posterior probability**, which is the updated probability of event E after observing evidence F.
- $P(F|E)$ is the **likelihood**, which is the probability of observing evidence F given that event E is true.
- $P(E)$ is the **prior probability**, which is the initial belief about the probability of event E before observing evidence F.
- $P(F)$ is the **marginal likelihood** or **evidence**, which is the total probability of observing evidence F under all possible events.

7.5 Beta Distribution

The **beta distribution** is a continuous probability distribution defined on the interval $[0, 1]$. The probability density function of a beta random variable $X \sim Beta(\alpha, \beta)$ is:

$$f_X(x) = \begin{cases} \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha,\beta)} & \text{for } 0 < x < 1 \\ 0 & \text{otherwise} \end{cases}$$

where B is the beta function, defined as:

$$B(\alpha, \beta) = \int_0^1 t^{\alpha-1}(1-t)^{\beta-1} dt$$

The beta distribution is flexible and can take various shapes depending on the values of α and β.
A beta distribution with $\alpha = \beta = 1$ is a uniform distribution on the interval $[0, 1]$.

The **expectation** of a beta random variable is given by:

$$E[X] = \frac{\alpha}{\alpha + \beta} \tag{18}$$

The **variance** of a beta random variable is given by:

$$Var(X) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)} \tag{19}$$

7.5.1 Experiment Design with Beta Distribution

Beta parameters α and β can come from an experiment as number of successes and failures, respectively. If we have observed s successes and f failures, we can set $\alpha = s + 1$ and $\beta = f + 1$ to get the posterior distribution of the probability of success. This is helpful because it allows us to update our beliefs about the probability of success as we collect more data.

We set +1 since we start from a uniform distribution, representing a state of complete uncertainty about the probability of success before observing any data. This represent a prior belief without any evidence, and allows us to incorporate the observed data in a way that reflects our updated beliefs about the probability of success.

If we have instead already experienced s successes and f failures previously, having a prior belief about the probability of success, we can set $\alpha = s + \alpha_0$ and $\beta = f + \beta_0$, where α_0 and β_0 are the parameters of our prior distribution. This creates a new posterior distribution that combines our prior belief with the observed data, allowing us to make more informed decisions.

Then if we want to calculate the probability that the random variables is greater than a certain value x, we can calculate the cumulative distribution function (CDF) of the beta distribution:

$$P(X > x) = 1 - F_X(x) = 1 - \int_0^x f_X(t)dt \tag{20}$$

This allows us to make probabilistic statements about the likelihood of the random variable exceeding a certain threshold.

7.6 Adding

In some cases, we may want to add two random variables together, for example, to calculate the total number of successes in two independent experiments. If A and B are two independent random variables, then the probability of the random variable C representing the sum of A and B can be calculated through the law of total probability:

$$P(C = k) = P(A + B = k) = \sum_{i=0}^{k} P(A = i)P(B = k - i) \tag{21}$$

The basic idea is to consider all possible ways that the sum C can equal k by summing over all possible values of A and B that add up to k.
However, if A and B are not discrete random variables, we would need to use the convolution of their probability density functions to calculate the distribution of their sum.

$$f_C(c) = \int_{-\infty}^{\infty} f_A(a)f_B(c - a)da \tag{22}$$

The idea is to integrate over all possible values of A and B that sum up to c, weighted by their respective probability densities.

Sometimes, we variables are **i.i.d.** (independent and identically distributed), meaning they have the same:

- All have the same distribution
- All have the same expectation
- All have the same variance

This simplifies the calculation of the distribution of their sum, since we can use the properties of the distribution to derive the distribution of the sum.

7.6.1 Sum of Foundational Random Variables

- **Sum of Binomial Random Variables**:
  $$X_1 \sim Bin(n_1, p), \quad X_2 \sim Bin(n_2, p) \implies X_1 + X_2 \sim Bin(n_1 + n_2, p)$$
- **Sum of Bernoulli Random Variables**:
  $$X_1 \sim Ber(p), \quad X_2 \sim Ber(p) \implies X_1 + X_2 \sim Bin(2, p)$$
- **Sum of Normal Random Variables**:
  $$X_1 \sim N(\mu_1, \sigma_1^2), \quad X_2 \sim N(\mu_2, \sigma_2^2) \implies X_1 + X_2 \sim N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$$

7.6.2 Central Limit Theorem

The **central limit theorem** states that the sum of a large number of independent and identically distributed random variables, each with finite mean and variance, will approximately follow a normal distribution, regardless of the original distribution of the variables.

$$\sum_{i=1}^{n} X_i \xrightarrow{d} N(n\mu, n\sigma^2) \quad \text{as } n \to \infty$$

This implies that the sum of idd random variables is always **normally distributed**, where the $E[X_i] = \mu$ and $Var(X_i) = \sigma^2$ for all i.

7.6.3 Sample Mean and Sample Variance

Given a sample of n independent and identically distributed random variables $X_1, X_2, ..., X_n$ with mean μ and variance σ^2, we can calculate the **sample mean** and **sample variance** as follows:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^{n} X_i \tag{23}$$

$$S^2 = \frac{1}{n-1} \sum_{i=1}^{n} (X_i - \bar{X})^2 \tag{24}$$

The sample mean $\bar{X}$ is an unbiased estimator of the population mean μ, and the sample variance S^2 is an unbiased estimator of the population variance σ^2.

7.7 Bootstraping

Bootrapping allows us to know the distribution of a statistic (e.g., mean, variance) and calculate p-values without making assumptions about the underlying distribution of the data.
The idea is that we can approximate the real distribution of a statistic by sampling directly from the data we have, and calculating the statistic for each sample. So we want to estimate the PMF of a statistic T trough this algorithm:

1. Repeat B times:
    - Draw a bootstrap sample of size n from the original data
    - Recalculate the statistic T^* for the bootstrap sample

In order to enhance we can use **resampling with replacement**, which means that we can draw the same data point multiple times in the bootstrap sample, increasing the data available for estimation.